

Deploying Qwen3-1.7B on iPhone with Apple Core AI: A Static W8 Implementation and Same-Device Profile Evaluation

Yi Shi

massif0601@gmail.com

ORCID: 0009-0002-7894-2703

Mengni Wu

mengninolab@gmail.com

ORCID: 0009-0004-7465-9067

August 28, 2026

Abstract

Apple’s public `coreai-models` repository contained no Qwen3-1.7B preset at the audited commit. We implement a version-pinned static iOS path with per-tensor W8 K-means-palettized transformer projections, a separately INT8-quantized tied embedding, FP16 compute, fixed KV state, a 4,096-token context, and **h16p** ahead-of-time compilation with Neural Engine-preferred compute. The selected W8 mechanism achieved 0.996598 mean and 0.959625 minimum logit cosine on a frozen holdout, 98.37% top-1 agreement, and a 0.003167 mean NLL delta relative to frozen uncompressed-reference outputs. On an iPhone 15 Pro running iOS 27, the historical W8 artifact completed four six-case suites. An Instruments trace recorded 547 MPSGraph program intervals and 547 Apple Neural Engine Prediction intervals, supporting ANE participation in the traced run. A separate public W8 rebuild also passed a six-case device suite. We compare this Static-W8 profile with a Dynamic-INT4 profile on the same device using a frozen 300-example CMRC2018 subset. Static-W8 scored 59.33 EM / 81.70 F1 and Dynamic-INT4 scored 57.67 EM / 81.42 F1; both paired confidence intervals included zero. Dynamic-INT4 used less storage and peak process memory and decoded faster, while Static-W8 reached the first token sooner and completed the near-4K prefill workload in less time. Because the profiles differ in several coupled design choices, these measurements compare deployable configurations rather than isolated precision or compute-unit effects.

1 Introduction

Deploying a language model on a phone requires more than compatible weights. Weight representation, KV-cache allocation, graph shape, compilation target, runtime sessions, and memory release jointly determine whether the model can be compiled and used on a physical device.

Qwen3 provides dense and mixture-of-experts language models with unified thinking and non-thinking behavior (Yang et al., 2025). This report concerns the dense Qwen3-1.7B checkpoint at a fixed upstream revision (Qwen Team, 2025). Both profiles studied here derive from revision 70d244cc86ccca08cf5af4e1e306ecf908b1ad5e and publish the same tokenizer SHA-256.

At Apple `coreai-models` commit 7062017c8e86c6cf4f49b721ddc3494efcdb7c7d, the Qwen3 support table and model registry listed 0.6B, 4B, and 8B presets but no 1.7B preset. Apple issue 116 remained an open model request at the August 28 audit (Apple Inc., 2026c). Pull request 196 proposed macOS and iOS support but closed without merge after its maintainer reported a local iOS regression; the comment did not disclose a root cause (Apple Inc., 2026d).

Community Core AI artifacts already cover dynamic GPU and stateful INT8 routes, with different devices, graph forms, validation methods, and published materials. We study whether Qwen3-1.7B can be added reproducibly to a static iOS authoring path, compiled for **h16p**, run through `LanguageModelSession`, and evaluated against a dynamic profile on the same device.

This report makes four contributions:

1. A version-pinned Apple-main patch, compression recipe, metadata, focused tests, export instructions, and **h16p** AOT procedure for an experimental Qwen3-1.7B iOS preset.

2. Frozen conversion-fidelity results, repeated physical-device suites, validation of the public W8 artifact, and an Instruments trace showing ANE participation.
3. A same-device evaluation against Dynamic-INT4 with per-example predictions, paired uncertainty estimates, per-run timing records, storage measurements, and process RSS.
4. Public recipes, a source patch, model artifacts, iOS sample applications, recorded hashes, and the deterministic pipeline used to reproduce the reported numbers.

2 Background and Related Work

2.1 Core AI authoring and runtime

Apple’s public `coreai-models` repository describes export recipes, reusable authoring primitives, and Swift runtime utilities for integrating `.aimodel` resources with Core AI on macOS and iOS (Apple Inc., 2026a). The repository requires version-27 operating systems and Xcode for the audited release and publishes a registry of model presets. Its public contribution policy at that commit also states that code pull requests are not being accepted at launch, so the patch in this work is presented as a reproducible reference implementation rather than an accepted upstream change.

Apple has separately documented transformer deployment on the Apple Neural Engine and model palettization (Apple Machine Learning Research, 2022; Apple Inc., 2026b). Those publications provide the general technical context for static-shape and compressed execution. ANE participation in our implementation is supported by the recorded Core AI trace.

2.2 Qwen3-1.7B public status

Table 1 separates Apple’s dated registry state, two community artifacts, and the two profiles evaluated in this work. The repository named `qwen3-1.7b-CoreAI-official` is a community Hugging Face repository, not an Apple publication. At its frozen model-card revision it reports a dynamic INT4 GPU bundle compiled for `h18p` and measurements on iPhone 17 Pro; those values are not reused as same-device results here (mlboydaisuke, 2026). A second community repository publishes a stateful INT8 `.aimodel`; its reproduction manifest uses the macOS export path, and its numeric accuracy gate is recorded as not run (kevinqz, 2026).

Work	Route	Public evidence	Boundary
Apple public Qwen3 registry	Official authoring presets	Qwen3-0.6B, 4B, and 8B; no 1.7B preset at the audited commit	Repository state only at commit 7062017c
Apple issue 116 / PR 196	Open request / closed attempt	Issue open; PR closed and unmerged after a reported local iOS regression	No regression root cause, acceptance, or roadmap commitment
mlboydaisuke community artifact	Dynamic INT4 / GPU / <code>h18p</code>	iPhone 17 Pro timing and eight model-card questions	Different device, context, and evaluation protocol
kevinqz community artifact	Stateful INT8 <code>.aimodel</code> ; macOS export manifest	Structural gate passed; numeric gate not run	No disclosed iPhone AOT or paired benchmark
This work: W8 profile	Static W8 / fixed KV / ANE-preferred / <code>h16p</code>	Frozen fidelity gate, iPhone 15 Pro suites, and ANE trace participation	One device class; no exclusive-ANE claim
This work: INT4 profile	Dynamic INT4 / growing KV / GPU-preferred / <code>h16p</code>	Same-device paired quality, latency, storage, and RSS comparison	Coupled system profile; no isolated causal attribution

Table 1: Public Qwen3-1.7B Core AI status audited 2026-08-28. Community model-card claims are contextual, not re-measured results.

Our evaluation focuses on a static `h16p` route on A17 Pro and a controlled comparison with a dynamic route on the same device.

3 System Profiles

3.1 Shared source identity

The W8 and INT4 artifacts originate from `Qwen/Qwen3-1.7B` revision `70d244cc86ccca08cf5af4e1e306ecf908b1ad5e`; their manifests record tokenizer SHA-256 `aeb13307a71acd8fe81861d94ad54ab689df773318809eed3cbe794b4492dae4`. Both cap the evaluated context at 4,096 tokens and target the `h16p` architecture, but most remaining deployment dimensions differ.

Dimension	Static-W8	Dynamic-INT4
Transformer weights	W8 per-tensor K-means palettization	INT4 block-32 symmetric-with-clipping
Embedding	INT8 per-tensor quantization	not separately reported in the artifact summary
Compute	FP16	FP16
KV cache	FP16 fixed-size	FP16 growing/dynamic
Graph	static	dynamic
Context cap	4,096 tokens	4,096 tokens
Preferred compute	neural-engine	gpu
AOT target	h16p	h16p

Table 2: Compared deployment profiles. The rows are coupled system choices, not independently randomized factors.

3.2 Static-W8 profile

Static-W8 begins from the unquantized upstream checkpoint. The recipe applies per-tensor, eight-bit K-means palettization to 196 transformer projection weights while excluding the embedding modules from that pass. The iOS authoring path quantizes the tied embedding separately to per-tensor INT8.

The reference patch adds a `Qwen3-1.7B` model preset and metadata, introduces the locked recipe, marks the preset experimental, adds focused registry and export-contract tests, and extends iOS conversion-matrix coverage. Against Apple main commit `04a3fd6cfe9bfae9cf05b1f246cf915d930d1c0a`, the patch applied cleanly; 28 focused tests passed, 42 related iOS conversion tests collected without errors, Ruff passed, and the dry-run resolved FP16 compute, the W8 recipe, 4,096 tokens, and enabled embedding quantization. The full export was not repeated on that later Apple commit because the relevant LLM authoring path was unchanged; compatibility on that commit is therefore claimed only for patch application, tests, lint, and preset resolution.

The exported `.aimodel` is compiled with `coreai-build` for iOS 27, `h16p`, and Neural Engine-preferred compute. The historical compiled artifact exposes 34 AOT functions across prompt, context, extension, and query buckets. The runtime loads the resource through `CoreAILanguageModel` and performs generation using `LanguageModelSession`.

Static W8 authoring-to-device path

Version-locked source, recipe, AOT target, runtime, and evidence

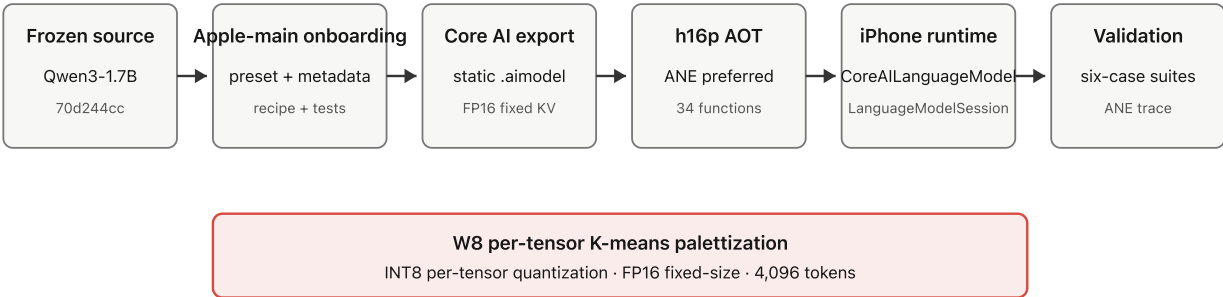


Figure 1: Version-pinned Static-W8 authoring-to-device path.

3.3 Dynamic-INT4 profile

Dynamic-INT4 uses block-32 symmetric-with-clipping INT4 transformer weights, FP16 compute, a growing FP16 KV cache, a dynamic graph, and GPU-preferred h16p compilation. On the test device, the runtime selected the `coreai-pipelined` engine and reported GPU compute.

An early pilot relied only on the `/no_think` prompt suffix. Inspection showed that this did not produce the same template state for both profiles, so the pilot was invalidated and its outputs were excluded. Before the accepted 300-pair run, the runtime was amended to pass `enable_thinking=false` to the model’s chat template; that control was then held constant across both profiles.

Coupled deployment profiles

The comparison does not isolate precision, graph shape, KV strategy, or compute unit causally

Static-W8		Dynamic-INT4	
Weights	W8 per-tensor K-means palettization	Weights	INT4 block-32 symmetric-with-clipping
Embedding	INT8 per-tensor quantization	Embedding	not separately reported in the artifact summary
KV state	FP16 fixed-size	KV state	FP16 growing/dynamic
Graph	static	Graph	dynamic
AOT	h16p; neural-engine	AOT	h16p; gpu

Shared: base revision, tokenizer, 4,096-token cap, h16p device class, iOS 27, and benchmark controls

Figure 2: The compared profiles bundle multiple choices. Performance differences cannot be attributed causally to ANE versus GPU or W8 versus INT4 alone.

4 Methods

4.1 Study design and provenance

The study protocol was frozen before manuscript drafting. The analysis uses existing public artifacts, predictions, per-run timing records, manifests, and trace summaries. It verifies the inputs listed in `analysis/source-lock.json`, reconstructs the frozen CMRC sample, reruns deterministic scoring, extracts the accepted timing samples, and generates the quantitative tables and figures. No additional model inference or device measurement is performed by the paper pipeline.

Every admitted artifact is identified by a repository commit or Hub revision and, where applicable, a SHA-256 digest. The current public W8 payload uses the same W8 mechanism but is not byte-identical to the W8 binary used in the historical A/B run. The historical comparison and the public rebuild therefore have separate identity and validation records.

4.2 W8 conversion-fidelity selection

W8 was selected with frozen uncompressed-reference logits and NLL rather than readable-text smoke tests. Uniform W4 and mixed W4/W8 candidates were rejected under the frozen conversion-fidelity contract before the W8 holdout was admitted; representative rejected mean-cosine values ranged from 0.856886 to 0.981232. The tuning and holdout sets remained separate. These measurements characterize conversion fidelity on the frozen inputs; they are not a downstream task benchmark.

4.3 Physical-device validation

The historical W8 artifact was tested on an iPhone 15 Pro (`iPhone16,1`, A17 Pro, `h16p`) running iOS 27. Each six-case suite covered structured output, multi-turn context, an instruction-injection boundary, a 3,790-token long-context retrieval, reasoning output, and unload/reload behavior. Four complete suites ran, and the final suite recorded zero hard failures.

An Instruments trace was summarized by counts of MPSGraph program and Apple Neural Engine Prediction intervals. The sanitized summary, rather than the original trace bundle, is public. The recorded intervals support ANE participation in the traced inference program.

The neutral-path public W8 rebuild was then validated separately on the same device class with a six-case suite. This second suite verifies the exact downloadable public payload but is not substituted for the historical A/B binary.

4.4 Paired CMRC2018 quality comparison

CMRC2018 is a span-extraction Chinese machine-reading-comprehension dataset (Cui et al., 2019). The paired experiment used a deterministic 300-example subset of its validation split: one question per unique context and 100 examples from each predefined context-length range (276–417, 418–630, and 631–980 characters), interleaved in a fixed order. Equal allocation gives the three length bands equal weight; it does not reproduce the validation split’s natural length distribution. The historical protocol records the cut points but no separate statistical derivation for them, so all inferences are limited to this frozen, length-balanced subset. Its SHA-256 is `9fb9d896c96fc2ef0fa9961a5b65d2ac5b8bd09744f717a0a0ecb9f6c9fe05ff`.

Both profiles used the same tokenizer, chat template with `enable_thinking=false`, prompt, order, 128-token response budget, temperature zero, and a fresh session per example. Raw generated text was retained. The paper pipeline rebuilt 3,219 canonical rows from official repository commit `c0eb1b6ba219847457e6af3180da722bbeb656af`, reconstructed the exact frozen sample, and reproduced the published Python-3 scoring JSON byte-for-byte under Python 3.11.15 and NLTK 3.10.0. The original CMRC scorer targeted Python 2; the reproduced result is the explicitly published Python-3 reanalysis.

The primary metrics use the official CMRC mixed Chinese/English EM/F1 procedure. Its `find_lcs` implementation scores the longest contiguous common token span. The analysis reports paired win/tie counts, 10,000-resample

paired bootstrap intervals, and two-sided paired sign tests (Efron, 1979). Intervals containing zero are reported as inconclusive; no equivalence or non-inferiority test was performed.

The initial feasibility protocol also required Dynamic-INT4 to remain within 1.0 point of Static-W8 on both EM and F1. The observed EM difference was -1.67 points, so this heuristic gate failed. The separately specified paired analysis yielded intervals that included zero.

4.5 Workload-specific latency and memory

The speed comparison evaluates complete deployment profiles rather than isolated hardware units. Both variants used the same tokenizer, chat template, prompts, non-thinking control, temperature, and fresh sessions. Each workload had one untimed warm-up followed by three accepted measured samples, with a distinct fixed-width tag to avoid an unreported repeat-input cache.

The workloads were: (1) 161 input / 60 output tokens for a compact structured task, (2) 3,790 input / 10 output tokens for near-4K prefill, and (3) 120 input / 256 output tokens for sustained decode. One Static-W8 generation terminated after 24 output tokens. Because the sustained-decode statistic was defined for runs reaching at least 200 tokens, this run was recorded, excluded from that statistic, and replaced. A separate diagnostic pass reused identical prompt tokens and showed cache-sensitive prefill timing; it was excluded before the accepted tagged samples were collected. Sustained-decode throughput is therefore conditional on reaching the protocol’s 200-token threshold.

Reported latency values are medians of the three accepted samples, not P95 estimates. Peak RSS means process resident memory observed in the disclosed benchmark, not total system RAM or a universal minimum-memory requirement.

5 Results

5.1 W8 fidelity and authoring validation

Table 3 reports the tuning and frozen-holdout conversion metrics. The holdout mean logit cosine was 0.996598, the minimum was 0.959625, top-1 agreement was 98.37%, and mean NLL delta was 0.003167.

Evaluation	Mean cosine	Min. cosine	Top-1 agree.	Mean Δ NLL
Tuning	0.997067	0.967261	98.44%	0.002985
Frozen holdout	0.996598	0.959625	98.37%	0.003167

Table 3: W8 conversion fidelity relative to frozen uncompressed-reference outputs. These are conversion metrics, not a capability benchmark.

The later Apple-main compatibility check established patch application, focused tests, related test collection, lint, and preset resolution. It did not repeat the full export on that newer commit and therefore does not claim binary identity with a newly exported payload.

5.2 Device execution and trace evidence

The historical artifact completed four full six-case suites. Its final suite loaded in 11.312 seconds after specialization-cache warm-up, peaked at 2,963.7 MiB process RSS, and returned to 198.7 MiB after unload. The long-context case consumed 3,790 input tokens, retrieved its tail marker, and produced a first token in 5.627 seconds.

The trace contained 547 MPSGraph program intervals and 547 Apple Neural Engine Prediction intervals. The exact public rebuild separately passed all six cases; it peaked at 2,766.4 MiB process RSS and produced the near-4K first token in 5.709 seconds.

Evidence object	Measure A	Measure B	Interpretation / identity
Historical suites	4 complete	6 per suite	Pass; zero recorded hard failures
Historical final suite	2,963.7 MiB peak RSS	198.7 MiB after unload	A-W8-HISTORICAL
Historical trace	547	547	ANE participation; non-exclusive
Public rebuild suite	6 / 6 cases	2,766.4 MiB peak RSS	A-W8-PUBLIC
Public near-4K case	3,790 input / 10 output	5.709 s TTFT	Tail marker retrieved

Table 4: Physical-device evidence on iPhone 15 Pro / iOS 27. Historical and public W8 payloads use the same mechanism but are not byte-identical.

5.3 Paired quality

All 300 examples completed for both profiles without hitting the 128-token budget. W8 produced 178 exact matches (59.33 EM) and 81.70 F1; INT4 produced 173 exact matches (57.67 EM) and 81.42 F1. The generated responses were byte-identical on 187 of 300 examples.

The observed Static-W8-minus-Dynamic-INT4 difference was +1.67 EM points with a 95% bootstrap interval of $[-3.00, +6.33]$ and +0.28 F1 points with an interval of $[-2.44, +3.00]$. The paired sign-test values were 0.576 for EM and 0.614 for F1. Both intervals include zero.

Profile	n	EM	F1	Exact matches
Static-W8	300	59.33	81.70	178
Dynamic-INT4	300	57.67	81.42	173
Static-W8 – Dynamic-INT4	300	+1.67	+0.28	–

W8-minus-INT4 95% bootstrap intervals: EM $[-3.00, +6.33]$, F1 $[-2.44, +3.00]$. Sign-test p : EM 0.576, F1 0.614.

Table 5: Paired CMRC2018 quality on the frozen 300-example subset. Both intervals include zero; no equivalence test was performed.

Paired quality difference: Static-W8 minus Dynamic-INT4

95% bootstrap intervals; both intervals cross zero

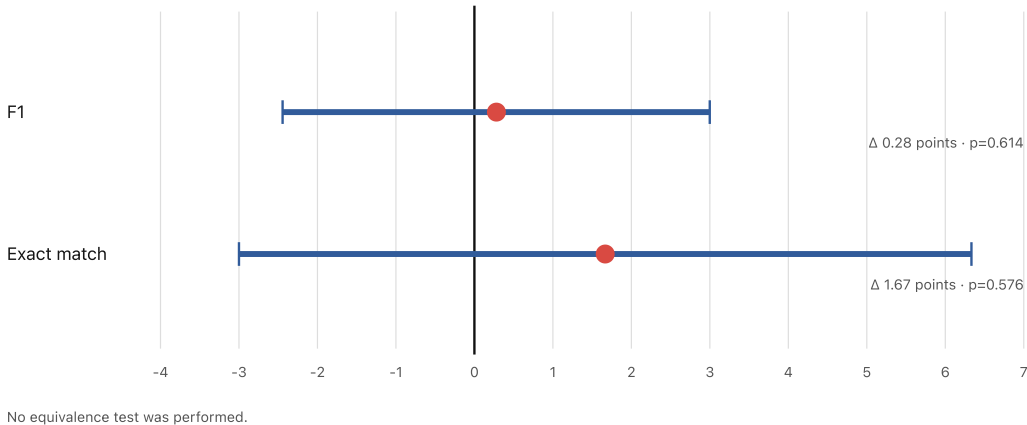


Figure 3: Paired Static-W8-minus-Dynamic-INT4 quality differences with 95% bootstrap intervals.

5.4 Storage and process memory

The historical W8 compiled resource directory occupied 1,714.4 MiB, compared with 924.6 MiB for INT4, a 789.8 MiB (46.1%) reduction. Peak process RSS was 2,865.0 MiB for W8 and 1,995.3 MiB for INT4, a reduction of 869.7 MiB (30.4%) in the disclosed benchmark.

Metric (MiB)	Static-W8	Dynamic-INT4	Dynamic-INT4 reduction
Compiled bundle	1,714.4	924.6	789.8 (46.1%)
Peak process RSS	2,865.0	1,995.3	869.7 (30.4%)

Table 6: Compiled resource-directory logical size and peak process resident memory in the disclosed benchmark.

5.5 Workload-dependent latency

For the 161/60 workload, W8 produced the first token in 0.431 seconds versus 1.221 seconds for INT4, but INT4 finished sooner (2.592 versus 3.259 seconds) and decoded faster (43.162 versus 20.505 visible tokens/s).

For the 3,790/10 workload, Static-W8 reduced TTFT from 13.522 to 5.647 seconds and total time from 13.717 to 6.116 seconds. Only nine visible decode tokens remained after excluding one reasoning token, so the decode-rate estimate is omitted.

For the 120/256 sustained-decode workload, INT4 reduced total time from 13.292 to 7.159 seconds and increased visible decode from 19.615 to 38.487 tokens/s. W8 retained the lower median TTFT, 0.404 versus 0.559 seconds.

Input / output	Profile	TTFT (s)	Total (s)	Visible decode (tok/s)
161 / 60	Static-W8	0.431	3.259	20.505
161 / 60	Dynamic-INT4	1.221	2.592	43.162
3,790 / 10	Static-W8	5.647	6.116	—
3,790 / 10	Dynamic-INT4	13.522	13.717	—
120 / 256	Static-W8	0.404	13.292	19.615
120 / 256	Dynamic-INT4	0.559	7.159	38.487

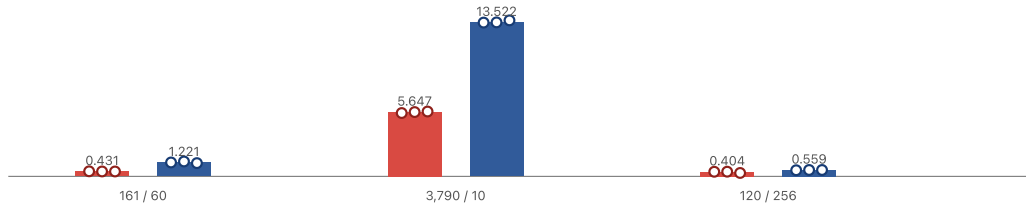
Table 7: Workload medians after one warm-up and three accepted samples. Near-4K decode rate is omitted because only nine visible tokens remained.

Workload-dependent latency

Input / output tokens; bars are medians and circles are individual accepted samples

Static-W8 Dynamic-INT4

Time to first token (seconds)



Total generation time (seconds)

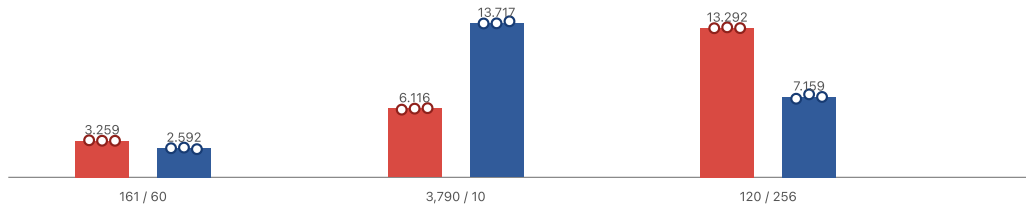


Figure 4: TTFT and total generation time for the three workload groups. Bars show medians; circles show the three accepted samples collected after one warm-up.

Storage and peak process memory

MiB; RSS is process resident memory, not total system RAM

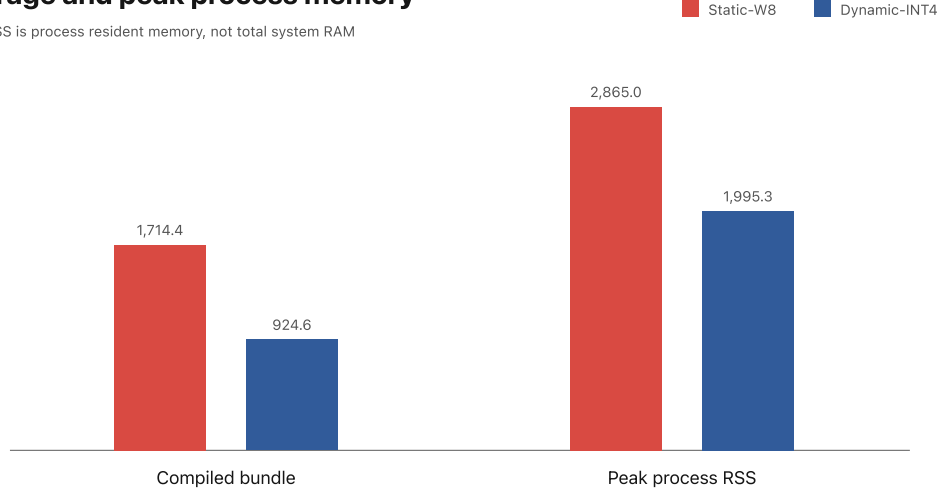


Figure 5: Compiled bundle size and peak process resident memory. RSS is not total system RAM.

Dynamic-INT4 used less storage and peak RSS and decoded faster in the sustained-generation workload. Static-W8 produced the first token sooner in all three workloads and completed the near-4K workload in less time.

6 Discussion

6.1 Why the winner changes with workload

The 161-token structured task combines a modest prefill with enough output for Dynamic-INT4’s higher decode rate to overcome its slower first token. The 3,790-token task reverses the balance: prefill dominates and Static-W8 reaches the first token more than twice as quickly. The 256-token task places more weight on sustained generation and favors Dynamic-INT4’s decode throughput. The relative performance of the two profiles therefore depends on the balance between prefill and generation in the measured workload.

Weight precision, embedding treatment, static versus dynamic shape, fixed versus growing KV, engine routing, artifact layout, and memory allocation all change together. The experiment cannot assign the observed differences to any one of these factors.

6.2 Implications for workload selection

Dynamic-INT4 performed better on the measured compact-prompt, long-output workload and used less storage and peak process RSS. Static-W8 performed better on the measured multi-thousand-token input with a short output. The two prompt lengths are insufficient to estimate a general crossover point.

CMRC2018 measures extractive Chinese reading comprehension. Transfer to summarization, classification, structured output, instruction boundaries, or bookmark analysis requires task-specific evaluation.

6.3 Reference implementation

The published reference implementation includes a tested Qwen3-1.7B preset, the selected compression recipe, an Apple-main patch, h16p compilation instructions, a Swift runtime session, physical-device test records, a sanitized trace summary, and downloadable model artifacts. The community GPU and stateful INT8 projects document two other deployment routes.

7 Reproducibility

The public W8 repository is frozen at commit `46668db811559ce21b0006f07706a6d0bde08656`; the paired-comparison repository is frozen at `34a4c08a9282bd076b8b5fe154c5507e6a8b3774` (Shi, 2026d,a). W8 empirical evidence is pinned to Hub revision `466ebe2e5cec125fa113ea71503add41bba581a8`. The current model-card revision `be6b5aad19e18bb71be3199eadceb27db7e69724` removes a local device nickname; the model payload and admitted validation file are unchanged. The INT4 Hub citation uses revision `c32b6342c98e5e23363f692e614bccca37f24234`; its predecessor `cb25b3226ee679ee92d0e5af7467d579a6bff66a` is the revision recorded by the original comparison provenance, and the later revision changes documentation while preserving the compiled payload hashes.(Shi, 2026c,b)

The W8 recipe SHA-256 is `dab03ae1dd6c6290a7964e05ebcda7fe027c2bb240174fcf034e64a376be9d72`. The public W8 compiled `main-h16p.mlirb` SHA-256 is `a7eefef16708a324f9919890355eb92180ec85eef419ebd5822e8c8afd42f5f`; the INT4 compiled `main` and `resources.bin` hashes are `ad953b6bc902acc1f1200a8870012c5dec6b488f0f4ec0f10a9916b16cb56ef` and `37c7f2fcf85625abb32d83ca24e7a9facd33a37f79dbc3afec64d41eb900f8bc`.

The repository separates evidence reconstruction from document compilation. Running `./analysis/run.sh` fetches the commits and remote objects listed in `analysis/source-lock.json`, verifies those inputs, reconstructs the CMRC reference sample, recomputes published quality, extracts per-run timing records, and regenerates Tables 1–7 and Figures 3–5. It pins Python 3.11.15 and NLTK 3.10.0. Running `./manuscript/build.sh` then checks the committed generated outputs and compiles the paper with the documented Chrome and Tectonic versions. The reconstructed quality JSON is byte-identical to the published Python-3 result.

The dataset text is not redistributed by the paper pipeline. Instead, the official CMRC source commit and SHA-256, canonical-row SHA-256, and frozen-sample SHA-256 are checked before scoring.

The downloadable W8 model is a later rebuild and cannot substitute for the historical A/B binary in an exact replay. The historical predictions, protocols, aggregate results, and recorded hashes remain available, and the public rebuild can be run independently.

8 Limitations and Threats to Validity

Hardware and toolchain. All device evidence is from one iPhone 15 Pro / A17 Pro / `h16p` class running iOS 27 with Xcode 27.0 build `27A5218g`. Other devices and later toolchains were not evaluated.

Coupled profiles. The system profiles differ in precision, compression granularity, embedding treatment, graph shape, KV allocation, preferred compute, and artifact layout. The evaluation therefore supports profile-level comparison, not causal attribution to ANE, GPU, W8, or INT4.

Execution trace. Only the sanitized trace summary is distributed. It records ANE participation but cannot be used to reconstruct event-level scheduling or exclusive dispatch.

Quality scope. The paired quality run contains a length-balanced subset of 300 CMRC2018 examples rather than the full validation split or its natural length distribution. Confidence intervals include zero, and no equivalence margin was specified. The initial 1.0-point EM heuristic failed. Complete WikiText-2 perplexity and downstream task evaluations are unavailable.

Timing scope. Each workload reports three accepted measured samples after one warm-up, which is insufficient for tail-latency estimates. Load observations were cache-sensitive. The sustained-decode comparison is conditional on generation reaching 200 output tokens, and the near-4K case contains too few visible decode tokens for a stable rate estimate.

Energy and lifecycle. Energy, battery use, thermal endurance, background execution, and Jetsam behavior were outside the measurement protocol.

Artifact identity. The downloadable W8 model uses the same mechanism as the historical A/B model, but it was rebuilt later. An exact replay still requires the historical binary; the published predictions, protocols, results, and hashes permit inspection of the recorded comparison.

9 Conclusion

We implemented a reproducible static Apple Core AI path for Qwen3-1.7B, compiled it for `h16p`, and ran it through `LanguageModelSession` on an iPhone 15 Pro with measured ANE participation. Static-W8 retained high conversion fidelity under the frozen inputs. In the same-device evaluation, Dynamic-INT4 used less storage and process memory and decoded faster, whereas Static-W8 reached the first token sooner and completed the long-prompt workload faster. The released materials allow these workload-dependent trade-offs to be inspected and measured again.

References

- Apple Inc. Core AI models. GitHub repository, commit 7062017c8e86c6cf4f49b721ddc3494efcdb7c7d, 2026a. URL <https://github.com/apple/coreai-models>. Accessed 2026-08-28.
- Apple Inc. Palettization overview: Guide to Core ML Tools. Core ML Tools documentation, 2026b. URL <https://apple.github.io/coremltools/docs-guides/source/opt-palettization-overview.html>. Accessed 2026-08-28.
- Apple Inc. Add Qwen3-1.7B support for iOS. GitHub issue 116, 2026c. URL <https://github.com/apple/coreai-models/issues/116>. State audited 2026-08-28.
- Apple Inc. Add Qwen3-1.7B iOS and macOS export support. GitHub pull request 196, 2026d. URL <https://github.com/apple/coreai-models/pull/196>. Closed without merge; state audited 2026-08-28.
- Apple Machine Learning Research. Deploying transformers on the Apple Neural Engine. Apple Machine Learning Research, 2022. URL <https://machinelearning.apple.com/research/neural-engine-transformers>.
- Yiming Cui, Ting Liu, Wanxiang Che, Li Xiao, Zhipeng Chen, Wentao Ma, Shijin Wang, and Guoping Hu. A span-extraction dataset for chinese machine reading comprehension. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 5883–5889, 2019. doi: 10.18653/v1/D19-1600. URL <https://aclanthology.org/D19-1600/>.
- Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.
- kevinqz. Qwen3-1.7B Core AI stateful INT8 artifact. Hugging Face model card, revision 340f4c4c8c422c56e0cc9b5ecb7eabdf37781f54, 2026. URL <https://huggingface.co/kevinqz/Qwen3-1.7B-CoreAI>. Community artifact; not published or endorsed by Apple.
- mlboydaisuke. Qwen3-1.7B Apple Core AI export for iPhone GPU. Hugging Face model card, revision af6335a8a108e383be8aa44925a8fe03a1cd2cd9, 2026. URL <https://huggingface.co/mlboydaisuke/qwen3-1.7b-CoreAI-official>. Community artifact; not published or endorsed by Apple.
- Qwen Team. Qwen3-1.7B. Hugging Face model repository, revision 70d244cc86ccca08cf5af4e1e306ecf908b1ad5e, 2025. URL <https://huggingface.co/Qwen/Qwen3-1.7B>.
- Yi Shi. Qwen3-1.7B Core AI reproduction and paired profile evaluation. GitHub repository, commit 34a4c08a9282bd076b8b5fe154c5507e6a8b3774, 2026a. URL <https://github.com/massif-01/qwen3-1.7b-coreai-reproduction>.

- Yi Shi. Qwen3-1.7B Core AI GPU INT4 4K h16p artifact. Hugging Face model repository, revision c32b6342c98e5e23363f692e614bccca37f24234, 2026b. URL <https://huggingface.co/massif/Qwen3-1.7B-CoreAI-GPU-INT4-4K-h16p>.
- Yi Shi. Qwen3-1.7B Core AI ANE W8 4K h16p artifact. Hugging Face model repository, revision be6b5aad19e18bb71be3199eadceb27db7e69724, 2026c. URL <https://huggingface.co/massif/Qwen3-1.7B-CoreAI-ANE-W8-4K-h16p>.
- Yi Shi. Qwen3-1.7B Core AI iOS: Reproducible W8 static-path implementation. GitHub repository, commit 46668db811559ce21b0006f07706a6d0bde08656, 2026d. URL <https://github.com/massif-01/qwen3-1.7b-coreai-ios>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.